LAB 10: Software Metrics & Complexity Analysis

Project: Digital Payment Gateway Simulation

Name: Ashutosh Tiwari | **Branch:** CSE, Section B, Semester V | **University Reg. No.:** 202401100200098 | **University Roll No.:** 2400290100098 | KIET Deemed to be University, Ghaziabad

1. Aim

To measure the size, complexity, coupling, cohesion and maintainability of the Digital Payment Gateway Simulation using standard software metrics, and to use the results to judge the quality of the design and decide where it should be improved.

2. Method and Tools

The metrics were found in two ways.

1. **From the design models** (the activity diagram, state transition diagram and data flow diagram prepared earlier): Function Point Analysis and cyclomatic complexity of the flow graph.
2. **From a reference implementation** of the same design, written in Python (`payment_gateway.py`, submitted along with this report). It contains nine classes: PaymentMethod, Card, UPI, AuthService, FraudDetector, BankSimulator, NotificationService, TransactionManager and PaymentGateway. The code metrics (LOC, cyclomatic complexity, Halstead measures and Maintainability Index) were measured with the open-source tool **radon**. The object-oriented metrics (WMC, DIT, NOC, CBO, RFC, LCOM) were calculated from the class structure.

Commands used to reproduce the code metrics:

Metric	Command
Raw size (LOC)	`radon raw payment_gateway.py`
Cyclomatic complexity	`radon cc -s -a payment_gateway.py`
Halstead measures	`radon hal payment_gateway.py`
Maintainability Index	`radon mi -s payment_gateway.py`

3. Metrics Used

Metric	What it measures	Formula or rule
LOC (physical, logical, source)	Size of the program	Count of lines
Function Points (FP)	Size from the user's point of view	FP = UFP x VAF, where VAF = 0.65 + 0.01 x TDI
Cyclomatic complexity V(G)	Number of independent paths, and so the minimum number of tests	V(G) = E - N + 2 = number of decisions + 1
Halstead measures	Size, difficulty and effort from operators and operands	Volume V = N log2(n); Difficulty D = (n1/2) x (N2/n2); Effort E = D x V
Maintainability Index (MI)	How easy the code is to maintain	Combines volume, complexity and LOC (radon scale 0 to 100)
WMC	Weighted methods per class	Sum of the cyclomatic complexity of the methods
DIT / NOC	Depth of inheritance tree / number of children	Counted from the class hierarchy
CBO	Coupling between objects	Number of other classes a class uses
RFC	Response for a class	Methods of the class plus the user-defined methods it calls
LCOM	Lack of cohesion of methods	Pairs of methods sharing no attribute minus pairs sharing one (minimum 0)

4. System Under Analysis

The system processes one payment in this order: validate payment details, authenticate the user (OTP), run the fraud and risk check, ask the bank simulator to debit the account, store the transaction, and send a notification. A successful payment can be refunded later. The gateway checks the merchant API key and the amount before handing the payment to the Transaction Manager.

5. Size Metrics

5.1 Lines of code (reference implementation)

Measure	Value
Physical lines (LOC)	124
Logical lines (LLOC)	98
Source lines (SLOC)	92
Blank lines	30
Comment lines	0
Docstring lines	2
Classes / methods	9 / 15

Observation: the program is small (about 92 source lines), but there are almost no comments. The comment ratio is 0%, which is a weakness for maintenance.

5.2 Function Point Analysis

Step 1: Count the function types and weigh them

Type	Item	Complexity	Weight
External Input (EI)	Order submission by the merchant	Average	4
EI	Payment details entry	High	6
EI	OTP / PIN entry	Low	3
EI	Refund request	Average	4
EI	Cancel payment	Low	3
External Output (EO)	Payment status to merchant	Average	5
EO	Receipt	Average	5
EO	SMS / e-mail notification	Low	4
EO	Failure message	Low	4
External Inquiry (EQ)	Transaction status	Low	3
EQ	Receipt view	Low	3
EQ	Refund status	Low	3
Internal Logical File (ILF)	User Accounts (data store D1)	Low	7
ILF	Fraud Rules (D2)	Low	7
ILF	Transactions (D3)	Average	10
External Interface File (EIF)	Bank Simulator account data	Average	7

Function type	Count	Total weight
EI	5	20
EO	4	18
EQ	3	9
ILF	3	24
EIF	1	7
Unadjusted Function Points (UFP)		**78**

Step 2: General System Characteristics (each rated 0 to 5)

No.	Characteristic	Rating	No.	Characteristic	Rating
1	Data communications	5	8	Online update	4
2	Distributed data processing	3	9	Complex processing	4
3	Performance	4	10	Reusability	3
4	Heavily used configuration	3	11	Installation ease	2
5	Transaction rate	4	12	Operational ease	3
6	Online data entry	5	13	Multiple sites	2
7	End-user efficiency	4	14	Facilitate change	4

Total Degree of Influence (TDI) = 5 + 3 + 4 + 3 + 4 + 5 + 4 + 4 + 4 + 3 + 2 + 3 + 2 + 4 = **50**

Step 3: Calculate

- Value Adjustment Factor: VAF = 0.65 + 0.01 x 50 = **1.15**
- Function Points: FP = UFP x VAF = 78 x 1.15 = **89.7, about 90 FP**
- Illustrative effort: assuming a productivity of 10 person-hours per FP, effort is about 897 person-hours, or roughly 5.6 person-months at 160 hours per month. The productivity figure is an assumption for illustration only.

6. Complexity Metrics

6.1 Cyclomatic complexity of the payment flow (from the activity diagram)

The payment workflow was converted into a control flow graph. The parallel steps (save transaction and send notification) were treated as one block.

Node	Meaning	Node	Meaning
1	Start	9	Decision: bank approves?
2	Select method and enter details	10	Save transaction and notify
3	Create order and send request	11	Return payment status
4	Authenticate user	12	Show result
5	Decision: auth valid?	13	Show failure message
6	Run fraud / risk check	14	View receipt / status
7	Decision: risk low?	15	End
8	Debit customer account		

Edges (17): 1-2, 2-3, 3-4, 4-5, 5-6, 5-13, 6-7, 7-8, 7-13, 8-9, 9-10, 9-13, 10-11, 11-12, 12-14, 13-14, 14-15.

- Nodes N = 15, Edges E = 17
- **V(G) = E - N + 2 = 17 - 15 + 2 = 4**
- Check by decisions: 3 decision nodes + 1 = 4

The four independent paths (minimum test cases):

Path	Route	Test scenario
P1	1-2-3-4-5-6-7-8-9-10-11-12-14-15	Successful payment
P2	1-2-3-4-5-13-14-15	Authentication fails
P3	1-2-3-4-5-6-7-13-14-15	Fraud flagged (risk high)
P4	1-2-3-4-5-6-7-8-9-13-14-15	Bank declines

A complexity of 4 means a simple, low-risk workflow.

6.2 Cyclomatic complexity of the code (radon)

Method	V(G)	Rank
TransactionManager.process_payment	5	A
FraudDetector.check_risk	3	A
TransactionManager.refund	3	A
PaymentGateway.create_payment	3	A
Card.validate	2	A
AuthService.authenticate	2	A
BankSimulator.debit	2	A
Eight other methods (constructors, simple checks)	1 each	A

- Sum of method complexities = 28 across 15 methods, so the **average is 28 / 15 = 1.87** per method.
- Radon's average over all 24 blocks (9 classes and 15 methods) is 2.21, which is grade A.
- The highest value is **process_payment with V(G) = 5**. It is one more than the 4 found in the activity diagram because the code also checks that the payment details are valid before authentication. The design model handled that check inside process 1.0.

The five test cases required for process_payment: invalid payment details, wrong OTP, high-risk payment, bank decline, and a successful payment.

6.3 Complexity of the transaction life cycle (state diagram)

The state transition diagram has 11 nodes (9 states, the initial state and the final state) and 14 transitions, so V(G) = 14 - 11 + 2 = **5**. The five independent paths are the successful payment, a cancelled payment, an authentication failure, a fraud decline or bank decline, and a refunded payment.

7. Halstead Metrics

Measured over the whole file with radon.

Symbol	Meaning	Value
n1	Distinct operators	13
n2	Distinct operands	36
N1	Total operators	20
N2	Total operands	38
n = n1 + n2	Vocabulary	49
N = N1 + N2	Program length	58
V = N log2(n)	Volume	325.65
D = (n1/2) x (N2/n2)	Difficulty	6.86
E = D x V	Effort	2,234.34
T = E / 18	Time to program (seconds)	124.13
B = V / 3000	Estimated delivered bugs	0.11

Check of the calculation: V = 58 x log2(49) = 58 x 5.615 = 325.65; D = (13 / 2) x (38 / 36) = 6.5 x 1.056 = 6.86; E = 6.86 x 325.65 = 2,234.

Observation: the volume and difficulty are low, so the code is small and easy to understand. The estimated bug count is about 0.11. Radon counts only arithmetic, comparison, boolean and similar operators, so these values describe the logic rather than every function call, and they should be read as indicators, not exact predictions.

8. Maintainability Index

The Maintainability Index of `payment_gateway.py` is **34.93**, which is **grade A** on radon's scale (A: above 19, B: 10 to 19, C: below 10).

The grade is good because complexity is low. The score is held down by the file length and the almost complete absence of comments. Adding explanatory comments would raise it.

9. Object-Oriented (CK) Metrics

Class	WMC	DIT	NOC	CBO	RFC	LCOM
PaymentMethod	1	0	2	0	1	0
Card	2	1	0	0	1	0
UPI	1	1	0	0	1	0
AuthService	3	0	0	0	2	0
FraudDetector	4	0	0	0	2	0
BankSimulator	3	0	0	0	2	0
NotificationService	1	0	0	0	1	0
TransactionManager	**9**	0	0	**5**	**8**	0
PaymentGateway	4	0	0	1	3	0
Total / average	28 / 3.11	max 1	max 2	6 / 0.67		all 0

Notes on how the values were found:

- CBO counts the other classes a class uses. TransactionManager uses AuthService, FraudDetector, BankSimulator, NotificationService and PaymentMethod, so its CBO is 5. Inheritance and standard library calls are not counted.
- RFC for TransactionManager is its 3 own methods plus the 5 methods it calls on other classes (validate, authenticate, check_risk, debit, send), which gives 8.
- LCOM is 0 for every class: the classes either have one working method or their methods share attributes (for example process_payment and refund both use `records`).

10. Interpretation

Metric	Result	Common guideline	Judgement
Cyclomatic complexity (highest)	5	10 or less is simple	Good
Average method complexity	1.87	Lower is better	Good
WMC (highest)	9	Below about 20	Good
DIT (highest)	1	Below about 5	Good, shallow hierarchy
NOC (highest)	2	Few children	Good
CBO (highest)	5	About 5 or less	Borderline for TransactionManager
LCOM	0	0 means cohesive	Good
Maintainability Index	34.93 (A)	Higher is better	Good, but lowered by missing comments
Function Points	about 90	Size indicator	Small to medium system

Overall: the design scores well on every measure. Every method is simple, the inheritance is shallow (only the PaymentMethod family), and the classes are cohesive. The one point to watch is the TransactionManager, which has the highest WMC (9), CBO (5) and RFC (8). This matches the design analysis, where the Transaction Manager was identified as the central hub of the system.

11. Recommendations

1. **Reduce the load on TransactionManager.** Split process_payment into small steps (validate, authenticate, risk check, debit) arranged as a pipeline, or let the notification service react to status-change events instead of being called directly. This lowers CBO and RFC.
2. **Write the five test cases** found in section 6.2 so that every independent path of process_payment is tested.
3. **Add comments and docstrings** to raise the Maintainability Index.
4. **Keep cyclomatic complexity below 10** as new payment methods and fraud rules are added. New payment methods should extend PaymentMethod, not add more branches to process_payment.
5. **Track the metrics after each change** so that growth in complexity is noticed early.

12. Conclusion

The analysis shows that the Digital Payment Gateway Simulation is a small and well-structured system of about 90 function points. Cyclomatic complexity is at most 5 for any method and 4 for the whole payment workflow, so the code is easy to test. The object-oriented metrics show shallow inheritance, high cohesion and low coupling, with the TransactionManager as the only class near a coupling limit. The Maintainability Index is in the highest grade. Splitting the TransactionManager and adding comments and tests would improve the design further.